2026–2027 PROFESSIONAL EDITION (VERSION 2.0)

Volume V: Proctored Mock Examination System

4 Full Certification Simulations, 73 Performance Tasks & Scoring Rubrics

Cloud Native Engineering Press

Targeting Kubernetes v1.31.x / v1.32.x (Forward Compatible through v1.33+)

Independent Professional Certification Study System

Certified Kubernetes Administrator (CNCF / The Linux Foundation)

VOLUME V: THE CKA MOCK EXAMINATION SYSTEM

CANDIDATE SCORECARD, GRADING RUBRICS, AND READINESS MATRIX

1. COMPREHENSIVE CANDIDATE SCORECARD

Use this master scorecard to calculate your score for each of the four mock examinations:

Domain	Weight	Points Max	Your Score
1. Troubleshooting	30%	30.0	
2. Cluster Architecture, Installation & Configuration	25%	25.0	
3. Services & Networking	20%	20.0	
4. Workloads & Scheduling	15%	15.0	
5. Storage	10%	10.0	
TOTAL COMPOSITE SCORE	100%	100.0	

$$\text{Final Percentage} = \frac{\text{Total Earned Points}}{\text{Total Possible Points}} \times 100$$

- **Passing Threshold: 66.0%**
- **Recommended Candidate Target: 80.0%+** (provides a safe buffer for PSI remote desktop latency).

2. SECONDARY COMPETENCY METRICS

Evaluation Dimension	Benchmark Target	Remediation Action if Below Benchmark
Command-Line Efficiency	≤ 30 seconds to generate any boilerplate YAML via `--dry-run=client -o yaml`.	Practice Section 74 speed setup drills daily.
Context Switching Hygiene	100% compliance; zero tasks executed in the wrong cluster context.	Tape a note to monitor: "CHECK CONTEXT FIRST".
Time Management	Complete 15 tasks within the first 90 minutes; 30 minutes for pass 3.	Enforce the 3-Pass Strategy (Pass 1: quick wins).
Troubleshooting Speed	Diagnose a `NotReady` node or crashed static pod in < 8 minutes.	Review Volume III diagnostic decision trees.

3. AUTOMATED VERIFICATION SCRIPT TEMPLATE

Candidates running local test clusters can execute this bash assertion script to automatically verify task completion:

```
#!/usr/bin/env bash
# Automated Verification Harness for CKA Mock Exam 4
set -uo pipefail

SCORE=0
MAX_SCORE=100

echo "================================================="
echo "STARTING AUTOMATED VERIFICATION AUDIT"
echo "================================================="

# Check Task 1: Node Ready
if kubectl get nodes node-2 | grep -q "Ready"; then
    echo "[PASS] Task 1: node-2 is Ready (+6)"
    SCORE=$((SCORE + 6))
else
    echo "[FAIL] Task 1: node-2 is NotReady"
fi

# Check Task 8: etcd snapshot exists
if [ -f /var/data/etcd-final-mock.db ]; then
    echo "[PASS] Task 8: etcd snapshot exists (+6)"
    SCORE=$((SCORE + 6))
else
    echo "[FAIL] Task 8: etcd snapshot missing"
fi

# Check Task 9: ServiceAccount & Role
if kubectl auth can-i create deployments -n operations --as=system:serviceaccount:operations:deployer | g
    echo "[PASS] Task 9: ServiceAccount permissions verified (+4)"
    SCORE=$((SCORE + 4))
else
    echo "[FAIL] Task 9: ServiceAccount cannot create deployments"
fi

# Check Task 19: PV Bound
if kubectl get pvc pvc-storage-1 2>/dev/null | grep -q "Bound"; then
    echo "[PASS] Task 19: PVC pvc-storage-1 is Bound (+5)"
    SCORE=$((SCORE + 5))
else
    echo "[FAIL] Task 19: PVC is not Bound"
fi

echo "================================================="
echo "TOTAL SCORE: ${SCORE} / ${MAX_SCORE}"
if [ $SCORE -ge 66 ]; then
    echo "STATUS: CANDIDATE PASSED"
else
    echo "STATUS: CANDIDATE FAILED (Threshold: 66)"
fi
echo "================================================="
```

4. POST-EXAM DIAGNOSTIC READINESS MATRIX

SCORE: < 55% [NOT READY - HIGH RISK]	SCORE: 55% – 65% [BORDERLINE - REVIEW REQUIRED]	SCORE: 66% – 100% [EXAM READY - EXCELLENT BUFFER]
• Do not sit the live exam yet. • Critical gaps in core triage and kubeadm commands. • Re-run Volume II practical workbooks for Domain 1 & 5.	• Review high-weight domains (Troubleshooting & Cluster Architecture). • Drill JSONPath queries and NetworkPolicies.	• Candidate exhibits necessary CLI muscle memory and triage instincts. • Execute a final timed mock exam 24 hours prior to sit.

VOLUME V: THE CKA MOCK EXAMINATION SYSTEM

MOCK EXAM 1: BEGINNER TO INTERMEDIATE SIMULATION

EXAMINATION SPECIFICATIONS

- **Target Difficulty:** Beginner to Intermediate
- **Time Allowed:** 120 minutes (2.0 hours)
- **Passing Score:** 66% (Total Possible: 100%)
- **Number of Tasks:** 17 Performance-Based Tasks
- **Cluster Contexts:** `k8s`, `cluster2`, `wk8s`

TASK 1: Pod Deployment and Label Filtering (Weight: 4%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Namespace `ingress-basic` does not exist.
- **Task Description:**
    1. Create namespace `ingress-basic`.
    2. Create a Pod named `frontend-pod` running image `nginx:1.27` in namespace `ingress-basic`.
    3. Assign labels `tier=frontend` and `app=web`.
    4. Expose container port `80`.
- **Expected Final Condition:** Pod `frontend-pod` is `Running` in namespace `ingress-basic` with both labels.
- **Verification Command:**

```
kubectl get pod frontend-pod -n ingress-basic --show-labels
```

- **Solution:**

```
kubectl config use-context k8s
kubectl create namespace ingress-basic
kubectl run frontend-pod --image=nginx:1.27 --port=80 --labels="tier=frontend,app=web" -n ingress-b
```

TASK 2: Multi-Container Pod with EmptyDir Sharing (Weight: 6%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Namespace `default`.
- **Task Description:**
  1. Create Pod `log-pipeline` in namespace `default`.
  2. Container 1 named `writer`: image `busybox:1.36`, appends `date` to `/var/log/app.log` every 5 seconds.
  3. Container 2 named `reader`: image `busybox:1.36`, executes `tail -f /var/log/app.log`.
  4. Both containers must mount an `emptyDir` volume at `/var/log`.
- **Expected Final Condition:** Both containers show `2/2 Running` and reader prints timestamps.
- **Verification Command:**

```
kubectl logs log-pipeline -c reader
```

- **Solution:**

```
apiVersion: v1
kind: Pod
metadata:
  name: log-pipeline
spec:
  volumes:
  - name: shared-vol
    emptyDir: {}
  containers:
  - name: writer
    image: busybox:1.36
    command: ["/bin/sh", "-c", "while true; do date >> /var/log/app.log; sleep 5; done"]
    volumeMounts:
    - name: shared-vol
      mountPath: /var/log
  - name: reader
    image: busybox:1.36
    command: ["/bin/sh", "-c", "tail -f /var/log/app.log"]
    volumeMounts:
    - name: shared-vol
      mountPath: /var/log
```

TASK 3: ServiceAccount and RBAC RoleBinding (Weight: 6%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Namespace `security-zone`.
- **Task Description:**
    1. Create namespace `security-zone`.
    2. Create a ServiceAccount named `operator-sa` in `security-zone`.
    3. Create a Role named `pod-admin` allowing `get`, `list`, `watch`, and `delete` on `pods`.
    4. Bind `pod-admin` to `operator-sa` via RoleBinding `operator-pod-bind`.
- **Expected Final Condition:** `operator-sa` can delete pods in `security-zone` but cannot delete deployments.
- **Verification Command:**

```
kubectl auth can-i delete pods -n security-zone --as=system:serviceaccount:security-zone:operator-s
```

- **Solution:**

```
kubectl create namespace security-zone
kubectl create serviceaccount operator-sa -n security-zone
kubectl create role pod-admin -n security-zone --verb=get,list,watch,delete --resource=pods
kubectl create rolebinding operator-pod-bind -n security-zone --role=pod-admin --serviceaccount=sec
```

TASK 4: Deployment Scaling and Rolling Update (Weight: 5%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Namespace `default`.
- **Task Description:**
    1. Create Deployment `micro-api` running `nginx:1.25` with 2 replicas.
    2. Scale the Deployment to 5 replicas.
    3. Update image to `nginx:1.26` and annotate with change-cause: "Updated to 1.26".
    4. Verify all 5 replicas are updated.
- **Expected Final Condition:** 5 replicas running image `nginx:1.26` with change-cause recorded.
- **Verification Command:**

```
kubectl rollout status deployment micro-api
kubectl rollout history deployment micro-api
```

- **Solution:**

```
kubectl create deployment micro-api --image=nginx:1.25 --replicas=2
kubectl scale deployment micro-api --replicas=5
kubectl set image deployment/micro-api nginx=nginx:1.26
kubectl annotate deployment/micro-api kubernetes.io/change-cause="Updated to 1.26" --overwrite
```

TASK 5: Secret Creation and Pod Environment Injection (Weight: 5%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Namespace `data-tier`.
- **Task Description:**
    1. Create namespace `data-tier`.
    2. Create Secret `db-passwords` containing `DB_PASS=AlpineP@ss2026`.
    3. Deploy Pod `db-runner` (`alpine`, sleeping 3600) injecting `DB_PASS` as an environment variable from Secret `db-passwords`.
- **Expected Final Condition:** `kubectl exec db-runner -- env` shows `DB_PASS=AlpineP@ss2026`.
- **Verification Command:**

```
kubectl exec db-runner -n data-tier -- env | grep DB_PASS
```

- **Solution:**

```
kubectl create namespace data-tier
kubectl create secret generic db-passwords --from-literal=DB_PASS='AlpineP@ss2026' -n data-tier
```

```
apiVersion: v1
kind: Pod
metadata:
  name: db-runner
  namespace: data-tier
spec:
  containers:
  - name: runner
    image: alpine
    command: ["sleep", "3600"]
    env:
    - name: DB_PASS
      valueFrom:
        secretKeyRef:
          name: db-passwords
          key: DB_PASS
```

TASK 6: PersistentVolume & PersistentVolumeClaim Binding (Weight: 7%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Host directory `/data/local-vol`.
- **Task Description:**
  1. Create PersistentVolume `pv-disk-1` with capacity `3Gi`, accessMode `ReadWriteOnce`, hostPath `/data/local-vol`, and storageClassName `local-disk`.
  2. Create PersistentVolumeClaim `pvc-disk-1` in `default` requesting `2Gi` on storageClassName `local-disk` with accessMode `ReadWriteOnce`.
  3. Confirm the claim status is `Bound`.
- **Expected Final Condition:** Both PV and PVC show status `Bound`.
- **Verification Command:**

```
kubectl get pvc pvc-disk-1
```

- **Solution:**

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-disk-1
spec:
  capacity:
    storage: 3Gi
  accessModes:
    - ReadWriteOnce
  storageClassName: local-disk
  hostPath:
    path: /data/local-vol
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-disk-1
spec:
  storageClassName: local-disk
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 2Gi
```

TASK 7: ClusterIP Service and DNS Discovery (Weight: 5%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Deployment `api-server` exists in `production`.
- **Task Description:**
    1. Create a ClusterIP Service named `api-svc` in `production` exposing Deployment `api-server` on port 80 targeting port 8080.
    2. Verify that EndpointSlices are created and contain valid IPs.
- **Expected Final Condition:** `api-svc` resolves internally and has active endpoints.
- **Verification Command:**

```
kubectl get endpoints api-svc -n production
```

- **Solution:**

```
kubectl expose deployment api-server --name=api-svc --port=80 --target-port=8080 -n production
```

TASK 8: Node Maintenance: Cordon and Safe Drain (Weight: 7%)

- **Context:** `kubectl config use-context cluster2`
- **Initial Condition:** Worker node `cluster2-worker-1` is running workloads.
- **Task Description:**
    1. Switch to context `cluster2`.
    2. Cordon `cluster2-worker-1`.
    3. Drain all workloads from `cluster2-worker-1` safely, handling DaemonSets and emptyDir volumes.
- **Expected Final Condition:** Node status shows `Ready,SchedulingDisabled` with 0 non-daemonset pods.
- **Verification Command:**

```
kubectl get node cluster2-worker-1
```

- **Solution:**

```
kubectl config use-context cluster2
kubectl cordon cluster2-worker-1
kubectl drain cluster2-worker-1 --ignore-daemonsets --delete-emptydir-data --force
```

TASK 9: Pod Scheduling with NodeSelector (Weight: 4%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Worker nodes untagged.
- **Task Description:**
    1. Label node `k8s-worker-1` with `disk=ssd`.
    2. Deploy Pod `cache-pod` running `redis:alpine` with a `nodeSelector` requiring `disk: ssd`.
    3. Verify Pod lands on `k8s-worker-1`.
- **Expected Final Condition:** `cache-pod` is `Running` on `k8s-worker-1`.
- **Verification Command:**

```
kubectl get pod cache-pod -o wide
```

- **Solution:**

```
kubectl config use-context k8s
kubectl label node k8s-worker-1 disk=ssd
kubectl run cache-pod --image=redis:alpine --dry-run=client -o yaml > cache.yaml
# Add nodeSelector: disk: ssd to spec
kubectl apply -f cache.yaml
```

TASK 10: Taints and Tolerations (Weight: 6%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Node `k8s-worker-2`.
- **Task Description:**
    1. Apply taint `dedicated=gpu:NoSchedule` to node `k8s-worker-2`.
    2. Deploy Pod `gpu-app` running `alpine` sleeping 3600 that tolerates `dedicated=gpu:NoSchedule`.
    3. Verify Pod schedules onto `k8s-worker-2`.
- **Expected Final Condition:** Pod is running on `k8s-worker-2`.
- **Verification Command:**

```
kubectl get pod gpu-app -o wide
```

- **Solution:**

```
kubectl taint nodes k8s-worker-2 dedicated=gpu:NoSchedule
```

```
apiVersion: v1
kind: Pod
metadata:
  name: gpu-app
spec:
  nodeName: k8s-worker-2
  tolerations:
  - key: "dedicated"
    operator: "Equal"
    value: "gpu"
    effect: "NoSchedule"
  containers:
  - name: app
    image: alpine
    command: ["sleep", "3600"]
```

TASK 11: Ingress Resource Path Routing (Weight: 7%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Ingress controller installed. Services `auth-svc` and `pay-svc` running on port 80.
- **Task Description:**
    1. Author an Ingress `gateway-ingress` in `default`.
    2. IngressClass: `nginx`. Host: `api.test.local`.
    3. Path `/auth` $\to$ `auth-svc:80`.
    4. Path `/pay` $\to$ `pay-svc:80`.
- **Expected Final Condition:** Ingress created with `Prefix` path matching.
- **Verification Command:**

```
kubectl describe ingress gateway-ingress
```

- **Solution:**

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: gateway-ingress
spec:
  ingressClassName: nginx
  rules:
  - host: api.test.local
    http:
      paths:
      - path: /auth
        pathType: Prefix
        backend:
          service:
            name: auth-svc
            port:
              number: 80
      - path: /pay
        pathType: Prefix
        backend:
          service:
            name: pay-svc
            port:
              number: 80
```

TASK 12: NetworkPolicy Restricting Ingress (Weight: 7%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Namespace `secure-backend`.
- **Task Description:**
  1. Create NetworkPolicy `isolate-backend` in `secure-backend` selecting pods labeled `role=db`.
  2. Permit incoming TCP traffic on port 5432 strictly from pods labeled `role=api`.
- **Expected Final Condition:** Non-api pods cannot reach port 5432.
- **Verification Command:**

```
kubectl describe netpol isolate-backend -n secure-backend
```

- **Solution:**

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: isolate-backend
  namespace: secure-backend
spec:
  podSelector:
    matchLabels:
      role: db
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          role: api
    ports:
    - protocol: TCP
      port: 5432
```

TASK 13: etcd Snapshot Backup (Weight: 8%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Control plane node.
- **Task Description:**
  1. SSH to control plane.
  2. Save etcd snapshot to `/opt/backup/etcd-mock1.db` using client certificates in `/etc/kubernetes/pki/etcd/`.
  3. Verify snapshot status.
- **Expected Final Condition:** Snapshot file exists with non-zero size and valid revision.
- **Verification Command:**

```
sudo ETCDCTL_API=3 etcdctl --write-out=table snapshot status /opt/backup/etcd-mock1.db
```

- **Solution:**

```
sudo ETCDCTL_API=3 etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  snapshot save /opt/backup/etcd-mock1.db
```

TASK 14: Gateway API HTTPRoute Deployment (Weight: 6%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Gateway `shared-gw` exists in `infra`. Service `order-svc` exists in `sales`.
- **Task Description:**
    1. In namespace `sales`, create an `HTTPRoute` named `order-route` attaching to parent Gateway `shared-gw` in namespace `infra`.
    2. Route path prefix `/orders` to Service `order-svc` on port 80.
- **Expected Final Condition:** `order-route` reports `Accepted: True`.
- **Verification Command:**

```
kubectl describe httproute order-route -n sales
```

- **Solution:**

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: order-route
  namespace: sales
spec:
  parentRefs:
  - name: shared-gw
    namespace: infra
  rules:
  - matches:
    - path:
        type: PathPrefix
        value: /orders
    backendRefs:
    - name: order-svc
      port: 80
```

TASK 15: Pod Liveness and Readiness Probe Configuration (Weight: 6%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Namespace `default`.
- **Task Description:**
    1. Create Pod `probe-app` running `nginx:1.27`.
    2. Add readiness probe: HTTP GET `/` on port 80, initialDelaySeconds 5.
    3. Add liveness probe: HTTP GET `/` on port 80, periodSeconds 10.

- **Expected Final Condition:** Pod is `1/1 Running`.
- **Verification Command:**

```
kubectl describe pod probe-app | grep -E "Liveness|Readiness"
```

- **Solution:**

```
apiVersion: v1
kind: Pod
metadata:
  name: probe-app
spec:
  containers:
  - name: nginx
    image: nginx:1.27
    readinessProbe:
      httpGet:
        path: /
        port: 80
      initialDelaySeconds: 5
    livenessProbe:
      httpGet:
        path: /
        port: 80
      periodSeconds: 10
```

TASK 16: JSONPath Extraction of Node Internal IPs (Weight: 4%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Multi-node cluster.
- **Task Description:**
    1. Extract the Internal IP of all nodes in the cluster using `kubectl get nodes -o jsonpath=...`.
    2. Save the output string to `/opt/node-ips.txt`.
- **Expected Final Condition:** `/opt/node-ips.txt` contains space-separated Internal IPs.
- **Verification Command:**

```
cat /opt/node-ips.txt
```

- **Solution:**

```
kubectl get nodes -o jsonpath='{.items[*].status.addresses[?(@.type=="InternalIP")].address}' > /op
```

TASK 17: Troubleshooting a CrashLoopBackOff Pod (Weight: 6%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Pod `broken-worker` in namespace `default` in `CrashLoopBackOff`.
- **Task Description:**
    1. Check previous logs of `broken-worker` to identify the crash reason.
    2. You discover the command `sh -c "sleep 10 && exit 1"` is failing.
    3. Fix the pod command to `sh -c "sleep 3600"`.
    4. Recreate the pod and verify it stays in `Running` status.
- **Expected Final Condition:** `broken-worker` is `Running` with 0 restarts.
- **Verification Command:**

```
kubectl get pod broken-worker
```

- **Solution:**

```
kubectl logs broken-worker --previous
kubectl get pod broken-worker -o yaml > /tmp/broken.yaml
sed -i 's/exit 1/sleep 3600/g' /tmp/broken.yaml
kubectl replace --force -f /tmp/broken.yaml
```

VOLUME V: THE CKA MOCK EXAMINATION SYSTEM

MOCK EXAM 2: INTERMEDIATE CLUSTER ADMINISTRATION SIMULATION

EXAMINATION SPECIFICATIONS

- **Target Difficulty:** Intermediate Administration
- **Time Allowed:** 120 minutes (2.0 hours)
- **Passing Score:** 66% (Total: 100%)
- **Number of Tasks:** 18 Performance-Based Tasks
- **Focus:** Cluster lifecycle, kubeadm, storage, traffic management, and workload administration.

TASK 1: Upgrade Control Plane via Kubeadm (Weight: 9%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Primary control plane node `cp-node` running Kubernetes v1.30.2.
- **Task Description:**
    1. Upgrade `kubeadm` package to version `1.31.2-1.1`.
    2. Plan and apply the upgrade using `kubeadm upgrade apply v1.31.2`.
    3. Drain `cp-node`.
    4. Upgrade `kubelet` and `kubectl` to `1.31.2-1.1`.
    5. Restart `kubelet` and uncordon `cp-node`.
- **Expected Final Condition:** Control plane node shows version `v1.31.2` in `Ready` status.
- **Solution:**

```
sudo apt-mark unhold kubeadm
sudo apt-get update && sudo apt-get install -y kubeadm=1.31.2-1.1
sudo apt-mark hold kubeadm
sudo kubeadm upgrade plan
sudo kubeadm upgrade apply v1.31.2 -y
kubectl drain $(hostname) --ignore-daemonsets --delete-emptydir-data
sudo apt-mark unhold kubelet kubectl
sudo apt-get install -y kubelet=1.31.2-1.1 kubectl=1.31.2-1.1
sudo apt-mark hold kubelet kubectl
sudo systemctl daemon-reload && sudo systemctl restart kubelet
kubectl uncordon $(hostname)
```

TASK 2: Upgrade Worker Node via Kubeadm (Weight: 7%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Worker node `worker-1` running v1.30.2.
- **Task Description:**
    1. Drain `worker-1` from the control plane.
    2. SSH to `worker-1`.
    3. Upgrade `kubeadm` to `1.31.2-1.1`.
    4. Run `sudo kubeadm upgrade node`.
    5. Upgrade `kubelet` and `kubectl` to `1.31.2-1.1`.
    6. Restart `kubelet` and uncordon from control plane.
- **Expected Final Condition:** `worker-1` runs `v1.31.2` in `Ready` status.
- **Solution:**

```
kubectl drain worker-1 --ignore-daemonsets --delete-emptydir-data
ssh worker-1
sudo apt-mark unhold kubeadm
sudo apt-get update && sudo apt-get install -y kubeadm=1.31.2-1.1
sudo apt-mark hold kubeadm
sudo kubeadm upgrade node
sudo apt-mark unhold kubelet kubectl
sudo apt-get install -y kubelet=1.31.2-1.1 kubectl=1.31.2-1.1
sudo apt-mark hold kubelet kubectl
sudo systemctl daemon-reload && sudo systemctl restart kubelet
exit
kubectl uncordon worker-1
```

TASK 3: Cluster Certificate Inspection & Renewal (Weight: 6%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Certificates in `/etc/kubernetes/pki/`.

- **Task Description:**
    1. Check expiration of all certificates using `kubeadm`.
    2. Renew `apiserver` certificate specifically.
    3. Verify the certificate validity is extended by 1 year.
- **Expected Final Condition:** `apiserver` cert shows updated expiration date.
- **Solution:**

```
sudo kubeadm certs check-expiration
sudo kubeadm certs renew apiserver
```

TASK 4: StatefulSet with VolumeClaimTemplates (Weight: 7%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Namespace `db-prod`.
- **Task Description:**
    1. Create namespace `db-prod`.
    2. Create Headless Service `mongo-hs` on port 27017.
    3. Create StatefulSet `mongo` with 2 replicas running `mongo:6.0`.
    4. Configure `volumeClaimTemplates` requesting 1Gi on storageClassName `standard` mounted at `/data/db`.
- **Expected Final Condition:** Pods `mongo-0` and `mongo-1` running with dedicated PVCs.
- **Solution:**

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mongo
  namespace: db-prod
spec:
  serviceName: mongo-hs
  replicas: 2
  selector:
    matchLabels:
      app: mongo
  template:
    metadata:
      labels:
        app: mongo
    spec:
      containers:
      - name: mongo
        image: mongo:6.0
        volumeMounts:
        - name: data
          mountPath: /data/db
  volumeClaimTemplates:
  - metadata:
      name: data
    spec:
      accessModes: [ "ReadWriteOnce" ]
      storageClassName: standard
      resources:
        requests:
          storage: 1Gi
```

TASK 5: Ingress with TLS Secret Termination (Weight: 6%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** TLS cert `/tmp/tls.crt` and key `/tmp/tls.key`. Service `web-svc` on port 80.
- **Task Description:**
    1. Create TLS Secret `web-tls` from cert and key files.
    2. Create Ingress `tls-ingress` terminating TLS for `secure.company.com` routing to `web-svc:80`.
- **Expected Final Condition:** Ingress configured with TLS termination.
- **Solution:**

```
kubectl create secret tls web-tls --cert=/tmp/tls.crt --key=/tmp/tls.key
```

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: tls-ingress
spec:
  ingressClassName: nginx
  tls:
  - hosts:
    - secure.company.com
    secretName: web-tls
  rules:
  - host: secure.company.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: web-svc
            port:
              number: 80
```

TASK 6: Gateway API Canary Traffic Split (Weight: 6%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Gateway `app-gateway` exists in `ingress-gw`.
- **Task Description:**
    1. In namespace `default`, create `HTTPRoute` named `split-route` attached to parent Gateway `app-gateway` in namespace `ingress-gw`.
    2. Route hostname `api.company.com`.
    3. Route 80% traffic to Service `api-v1` on port 80, and 20% traffic to Service `api-v2` on port 80.
- **Expected Final Condition:** HTTPRoute reports `Accepted: True`.
- **Solution:**

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: split-route
  namespace: default
spec:
  parentRefs:
  - name: app-gateway
    namespace: ingress-gw
  hostnames:
  - "api.company.com"
  rules:
  - backendRefs:
    - name: api-v1
      port: 80
      weight: 80
    - name: api-v2
      port: 80
      weight: 20
```

TASK 7: StorageClass with WaitForFirstConsumer (Weight: 5%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Default storage provisioner active.
- **Task Description:**
  1. Create StorageClass `topology-sc` with `volumeBindingMode: WaitForFirstConsumer` and `reclaimPolicy: Delete`.
  2. Create PVC `topo-pvc` requesting 2Gi on `topology-sc`.
  3. Verify PVC remains in `Pending` until consumed by a pod.
- **Expected Final Condition:** PVC stays Pending with `WaitForFirstConsumer` condition.
- **Solution:**

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: topology-sc
provisioner: rancher.io/local-path
volumeBindingMode: WaitForFirstConsumer
reclaimPolicy: Delete
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: topo-pvc
spec:
  storageClassName: topology-sc
  accessModes: [ "ReadWriteOnce" ]
  resources:
    requests:
      storage: 2Gi
```

TASK 8: Helm Release Lifecycle and Override (Weight: 5%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Helm installed.
- **Task Description:**
  1. Add repo `bitnami` (`https://charts.bitnami.com/bitnami`).
  2. Install release `web-prod` using chart `bitnami/nginx` into namespace `production` .
  3. Override replica count to `3` using `--set replicaCount=3` .
- **Expected Final Condition:** Release `web-prod` deployed with 3 pods.
- **Solution:**

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
helm install web-prod bitnami/nginx -n production --create-namespace --set replicaCount=3
```

TASK 9: Declarative Kustomize Deployment (Weight: 5%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Kustomize directory `/opt/k8s/overlays/dev/` .
- **Task Description:**
  1. Inspect `/opt/k8s/overlays/dev/kustomization.yaml` .
  2. Apply the kustomization directly to the cluster without intermediate files.
- **Expected Final Condition:** All resources rendered and applied in target namespace.

- **Solution:**

```
kubectl apply -k /opt/k8s/overlays/dev/
```

TASK 10: Pod PriorityClass and Preemption (Weight: 5%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Namespace `default`.
- **Task Description:**
    1. Create a `PriorityClass` named `critical-tier` with value `1000000` and `preemptionPolicy: PreemptLowerPriority`.
    2. Deploy Pod `core-auth` using `priorityClassName: critical-tier`.
- **Expected Final Condition:** Pod runs with priority class `critical-tier`.
- **Solution:**

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: critical-tier
value: 1000000
preemptionPolicy: PreemptLowerPriority
---
apiVersion: v1
kind: Pod
metadata:
  name: core-auth
spec:
  priorityClassName: critical-tier
  containers:
  - name: auth
    image: alpine
    command: ["sleep", "3600"]
```

TASK 11: Horizontal Pod Autoscaler (HPA) (Weight: 6%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Deployment `api-worker` has CPU requests defined.
- **Task Description:**
    1. Configure an HPA named `api-worker-hpa` targeting Deployment `api-worker`.
    2. Min replicas: `2`, Max replicas: `10`, Target CPU utilization: `70%`.
- **Expected Final Condition:** HPA active and tracking metric utilization.
- **Solution:**

```
kubectl autoscale deployment api-worker --name=api-worker-hpa --cpu-percent=70 --min=2 --max=10
```

TASK 12: Interlocking ResourceQuota and LimitRange (Weight: 6%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Namespace `dev-team`.
- **Task Description:**
  1. Create namespace `dev-team`.
  2. Create ResourceQuota `team-quota` with `hard.pods: "5"` and `hard.requests.cpu: "2"`.
  3. Create LimitRange `team-limit` with container defaultRequest `cpu: 100m` and default `cpu: 200m`.
- **Expected Final Condition:** LimitRange injects defaults and satisfies quota automatically.
- **Solution:**

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: team-quota
  namespace: dev-team
spec:
  hard:
    pods: "5"
    requests.cpu: "2"
---
apiVersion: v1
kind: LimitRange
metadata:
  name: team-limit
  namespace: dev-team
spec:
  limits:
  - default:
      cpu: 200m
    defaultRequest:
      cpu: 100m
    type: Container
```

TASK 13: Native Sidecar in InitContainers (Weight: 5%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Namespace `default`.
- **Task Description:**

1. Author Pod `sidecar-app` with an initContainer `log-shipper` (`busybox:1.36` , `command: ["sleep", "3600"]`) and `restartPolicy: Always` .
2. Main container `web` running `nginx:1.27` .
3. Verify both containers run concurrently (`2/2`).

- **Expected Final Condition:** Pod shows `READY: 2/2` .
- **Solution:**

```
apiVersion: v1
kind: Pod
metadata:
  name: sidecar-app
spec:
  initContainers:
  - name: log-shipper
    image: busybox:1.36
    command: ["sleep", "3600"]
    restartPolicy: Always
  containers:
  - name: web
    image: nginx:1.27
```

TASK 14: NetworkPolicy Cross-Namespace Filtering (Weight: 6%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Namespace `db-space` .
- **Task Description:**

1. In namespace `db-space` , create NetworkPolicy `allow-web-ns` selecting pods `role=mysql` .
2. Allow Ingress on port 3306 strictly from namespaces labeled `access=granted` .

- **Expected Final Condition:** Traffic to port 3306 is allowed only from namespaces labeled `access=granted` .
- **Solution:**

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-web-ns
  namespace: db-space
spec:
  podSelector:
    matchLabels:
      role: mysql
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          access: granted
    ports:
    - protocol: TCP
      port: 3306
```

TASK 15: Troubleshoot Empty Service Endpoints (Weight: 5%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Service `web-svc` has `ENDPOINTS: <none>`.
- **Task Description:**
    1. Identify why Service `web-svc` in `default` has no endpoints.
    2. Fix the label selector so endpoints reflect the running pods.
- **Expected Final Condition:** Service endpoints are populated.
- **Solution:**

```
kubectl get svc web-svc -o yaml
kubectl get pods --show-labels
# Change selector in Service to match pod labels
kubectl patch svc web-svc -p '{"spec":{"selector":{"app":"frontend"}}}'
```

TASK 16: Troubleshoot ImagePullBackOff (Weight: 4%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Pod `broken-pull` showing `ImagePullBackOff`.
- **Task Description:**
    1. Inspect pod events.
    2. Fix typo in image tag (`nginx:1.277` $\to$ `nginx:1.27`).
- **Expected Final Condition:** Pod transitions to `Running`.

- **Solution:**

```
kubectl set image pod broken-pull broken-pull=nginx:1.27
```

TASK 17: Containerd Low-Level Process Inspection (Weight: 4%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Worker node `worker-1`.
- **Task Description:**
    1. SSH to `worker-1`.
    2. Use `crictl` to list all running containers.
    3. Extract container ID of the `kube-proxy` container and output logs to `/tmp/proxy.log`.
- **Expected Final Condition:** `/tmp/proxy.log` contains containerd runtime logs.
- **Solution:**

```
ssh worker-1
sudo crictl ps --name=kube-proxy
CID=$(sudo crictl ps --name=kube-proxy -q)
sudo crictl logs $CID > /tmp/proxy.log
```

TASK 18: Extract Pod Restart Counts Sorted with JSONPath (Weight: 3%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Namespace `kube-system`.
- **Task Description:**
    1. Sort all pods in `kube-system` by `.status.containerStatuses[0].restartCount`.
    2. Write Pod names and restart counts to `/opt/restarts.txt`.
- **Expected Final Condition:** Tabular text file sorted by restart frequency.
- **Solution:**

```
kubectl get pods -n kube-system --sort-by='.status.containerStatuses[0].restartCount' -o custom-col
```

VOLUME V: THE CKA MOCK EXAMINATION SYSTEM

MOCK EXAM 3: ADVANCED TROUBLESHOOTING SIMULATION

EXAMINATION SPECIFICATIONS

- **Target Difficulty:** Advanced Troubleshooting (High Stress)
- **Time Allowed:** 120 minutes (2.0 hours)
- **Passing Score:** 66% (Total: 100%)
- **Number of Tasks:** 18 Performance-Based Diagnostic Scenarios
- **Focus:** Cluster component failure, node outages, broken manifests, storage locks, etcd recovery, and networking drop chains.

TASK 1: Recover Inactive Kubelet Due to Linux Swap (Weight: 6%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Worker node `worker-1` shows `NotReady`.
- **Task Description:**
    1. SSH to `worker-1`.
    2. Inspect why `systemctl status kubelet` reports failed status.
    3. Journalctl indicates swap is active.
    4. Disable swap immediately, disable it in `/etc/fstab`, and start `kubelet`.
- **Expected Final Condition:** `worker-1` returns to `Ready` status.
- **Solution:**

```
ssh worker-1
sudo swapoff -a
sudo sed -i '/swap/s/^/#/' /etc/fstab
sudo systemctl restart kubelet
sudo systemctl is-active kubelet
exit
kubectl get nodes worker-1
```

TASK 2: Kubelet Cgroup Driver Mismatch Recovery (Weight: 6%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Worker node `worker-2` fails to boot kubelet.
- **Task Description:**
    1. SSH to `worker-2`.
    2. Inspect kubelet journal logs for cgroup error.
    3. Align `/var/lib/kubelet/config.yaml` to use `cgroupDriver: systemd`.
    4. Restart kubelet.
- **Expected Final Condition:** Kubelet active and node `worker-2` shows `Ready`.
- **Solution:**

```
ssh worker-2
sudo sed -i 's/cgroupDriver: cgroupfs/cgroupDriver: systemd/g' /var/lib/kubelet/config.yaml
sudo systemctl restart kubelet
exit
kubectl get nodes worker-2
```

TASK 3: Repair Broken API Server Static Pod Flag (Weight: 7%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** `kubectl` returns connection refused on port 6443.
- **Task Description:**
    1. SSH to control plane node.
    2. Use `crictl ps -a` to find the exited `kube-apiserver` container.
    3. View container logs using `crictl logs`.
    4. Notice error: `unknown flag: --etcd-cafile-broken`.
    5. Edit `/etc/kubernetes/manifests/kube-apiserver.yaml` and fix flag to `--etcd-cafile=/etc/kubernetes/pki/etcd/ca.crt`.
    6. Wait for kubelet to relaunch container.
- **Expected Final Condition:** API server boots and `kubectl get nodes` succeeds.
- **Solution:**

```
sudo crictl ps -a --name=kube-apiserver
sudo sed -i 's/--etcd-cafile-broken/--etcd-cafile/g' /etc/kubernetes/manifests/kube-apiserver.yaml
sleep 15
kubectl get nodes
```

TASK 4: Repair Crashed Kube-Scheduler Static Pod (Weight: 5%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** All new pods stuck in `Pending` with 0 scheduler events.
- **Task Description:**
    1. Notice `kube-scheduler` static pod is not running in `kube-system`.
    2. Inspect `/etc/kubernetes/manifests/kube-scheduler.yaml` on control plane.
    3. Fix typo in container command: `kube-schedulerr` $\to$ `kube-scheduler`.
- **Expected Final Condition:** Scheduler launches and pending pods are scheduled.
- **Solution:**

```
sudo sed -i 's/kube-schedulerr/kube-scheduler/g' /etc/kubernetes/manifests/kube-scheduler.yaml
```

TASK 5: etcd Defragmentation and Space Alarm Disarm (Weight: 6%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** API server returns `etcdserver: mvcc: database space exceeded`.
- **Task Description:**
    1. Compact etcd history to current revision.
    2. Defragment etcd storage.
    3. Disarm `NOSPACE` alarm.
- **Expected Final Condition:** Cluster writes succeed.
- **Solution:**

```
export ETCDCTL_API=3
export CERTS="--cacert=/etc/kubernetes/pki/etcd/ca.crt --cert=/etc/kubernetes/pki/etcd/server.crt -
REV=$(sudo etcdctl $CERTS endpoint status -w json | jq .[0].Status.header.revision)
sudo etcdctl $CERTS compact $REV
sudo etcdctl $CERTS defrag
sudo etcdctl $CERTS alarm disarm
```

TASK 6: etcd Point-in-Time Snapshot Restore (Weight: 8%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Backup exists at `/opt/backup/etcd.db`.
- **Task Description:**
    1. Stop static pods by moving manifests out of `/etc/kubernetes/manifests/`.
    2. Restore snapshot to `/var/lib/etcd-recovery`.
    3. Update `etcd.yaml` to point to `/var/lib/etcd-recovery`.
    4. Move manifests back and verify cluster state.
- **Expected Final Condition:** Cluster recovers from snapshot.
- **Solution:**

```
sudo mv /etc/kubernetes/manifests/kube-apiserver.yaml /tmp/
sudo mv /etc/kubernetes/manifests/etcd.yaml /tmp/
sudo ETCDCTL_API=3 etcdctl --data-dir=/var/lib/etcd-recovery snapshot restore /opt/backup/etcd.db
sudo sed -i 's|/var/lib/etcd|/var/lib/etcd-recovery|g' /tmp/etcd.yaml
sudo mv /tmp/etcd.yaml /etc/kubernetes/manifests/
sudo mv /tmp/kube-apiserver.yaml /etc/kubernetes/manifests/
sleep 15
kubectl get nodes
```

TASK 7: CoreDNS Loop Crash Recovery (Weight: 5%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** CoreDNS in `CrashLoopBackOff` with loop error.
- **Task Description:**
    1. Inspect CoreDNS logs.
    2. Update kubelet config `/var/lib/kubelet/config.yaml` to set `resolvConf: /run/systemd/resolve/resolv.conf`.
    3. Restart kubelet and delete crash-looping CoreDNS pods.
- **Expected Final Condition:** CoreDNS pods enter `Running` status.
- **Solution:**

```
sudo sed -i 's|resolvConf: .*|resolvConf: /run/systemd/resolve/resolv.conf|g' /var/lib/kubelet/conf
sudo systemctl restart kubelet
kubectl delete pods -n kube-system -l k8s-app=kube-dns
```

TASK 8: Enable Node IP Forwarding in Linux Kernel (Weight: 5%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Pods on `worker-1` cannot reach external network.
- **Task Description:**
    1. SSH to `worker-1`.
    2. Check `sysctl net.ipv4.ip_forward`. Output is 0.
    3. Enable IP forwarding dynamically and persist in `/etc/sysctl.d/k8s.conf`.
- **Expected Final Condition:** `net.ipv4.ip_forward = 1` and cross-node traffic flows.
- **Solution:**

```
ssh worker-1
sudo sysctl -w net.ipv4.ip_forward=1
echo "net.ipv4.ip_forward = 1" | sudo tee -a /etc/sysctl.d/k8s.conf
```

TASK 9: Fix NetworkPolicy Blocking Outbound DNS (Weight: 6%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Pods in `secure-zone` cannot resolve domain names.
- **Task Description:**
    1. Inspect egress NetworkPolicy `egress-lock` in `secure-zone`.
    2. Notice it lacks port 53 UDP/TCP egress rules.
    3. Add an egress rule allowing port 53 to all destinations.
- **Expected Final Condition:** DNS resolution works inside `secure-zone`.
- **Solution:**

```
# Patch egress-lock to include:
- ports:
  - protocol: UDP
    port: 53
  - protocol: TCP
    port: 53
```

TASK 10: Triage OOMKilled Container (Exit Code 137) (Weight: 5%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Deployment `report-gen` crashing with OOMKilled.
- **Task Description:**

1. Inspect container termination status.
2. Increase memory limit from `64Mi` to `256Mi`.

- **Expected Final Condition:** Deployment pods stabilize at `Running`.
- **Solution:**

```
kubectl set resources deployment report-gen --limits=memory=256Mi --requests=memory=128Mi
```

TASK 11: Fix Pod CrashLoopBackOff Due to Missing ConfigMap (Weight: 5%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Pod `worker-app` showing `CreateContainerConfigError`.
- **Task Description:**
  1. Inspect pod events. Missing ConfigMap `app-settings`.
  2. Create ConfigMap `app-settings` with key `mode=prod`.
- **Expected Final Condition:** Pod transitions to `Running`.
- **Solution:**

```
kubectl create configmap app-settings --from-literal=mode=prod
```

TASK 12: Admission Webhook Deadlock Recovery (Weight: 6%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** All resource creation requests hang for 30s with webhook timeout.
- **Task Description:**
  1. Inspect validating webhooks: `kubectl get validatingwebhookconfigurations`.
  2. Locate failed webhook `validator.corp.local`.
  3. Delete the blocking webhook configuration to unfreeze the cluster.
- **Expected Final Condition:** API server responds immediately.
- **Solution:**

```
kubectl delete validatingwebhookconfiguration validator.corp.local
```

TASK 13: Reclaim Released PV by Removing ClaimRef (Weight: 5%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** PV `pv-legacy` shows `Released`.
- **Task Description:**
    1. Clear `spec.claimRef` from PV `pv-legacy`.
    2. Verify status transitions to `Available`.
- **Expected Final Condition:** PV is `Available` for new PVC binding.
- **Solution:**

```
kubectl patch pv pv-legacy -p '{"spec":{"claimRef": null}}'
```

TASK 14: Fix Failing LivenessProbe Restart Loop (Weight: 5%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Pod `web-probe` restarts every 30 seconds.
- **Task Description:**
    1. Inspect probe failure events.
    2. Fix probe path from `/healthz-wrong` to `/` on port 80.
- **Expected Final Condition:** Pod remains `Running` with 0 restarts.
- **Solution:**

```
kubectl get pod web-probe -o yaml > /tmp/probe.yaml
sed -i 's|/healthz-wrong|/|g' /tmp/probe.yaml
kubectl replace --force -f /tmp/probe.yaml
```

TASK 15: Fix DaemonSet Missing Control Plane Toleration (Weight: 4%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** DaemonSet `node-diag` does not schedule on control plane.
- **Task Description:**
    1. Add toleration for `node-role.kubernetes.io/control-plane:NoSchedule` to DaemonSet spec.
- **Expected Final Condition:** Pod schedules onto the control plane node.
- **Solution:**

```
# Add to spec.template.spec.tolerations:
- key: node-role.kubernetes.io/control-plane
  operator: Exists
  effect: NoSchedule
```

TASK 16: Clear Worker Node DiskPressure Eviction (Weight: 5%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Node `worker-1` shows `DiskPressure: True`.
- **Task Description:**
    1. SSH to `worker-1`.
    2. Use `crictl rmi --prune` to remove unused images and clean dangling logs in `/var/log/pods`.
- **Expected Final Condition:** `DiskPressure` transitions to `False`.
- **Solution:**

```
ssh worker-1
sudo crictl rmi --prune
sudo rm -rf /var/log/pods/deleted-*
```

TASK 17: Service TargetPort Mismatch Triage (Weight: 4%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Service `web-svc` has endpoints, but curling returns connection refused.
- **Task Description:**
    1. Discover `web-svc` targetPort is set to 8080, but NGINX listens on 80.
    2. Update `targetPort: 80`.
- **Expected Final Condition:** Curling Service returns HTTP 200.
- **Solution:**

```
kubectl patch svc web-svc -p '{"spec":{"ports":[{"port":80,"targetPort":80}]}}'
```

TASK 18: Extract Node Storage Driver via Custom Columns (Weight: 2%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Multi-node cluster.

- **Task Description:**
    1. List node names and container runtime versions using custom-columns.
    2. Write output to `/tmp/runtimes.txt`.
- **Expected Final Condition:** Text file populated with custom headers.
- **Solution:**

```
kubectl get nodes -o custom-columns=NAME:.metadata.name,RUNTIME:.status.nodeInfo.containerRuntimeVe
```

VOLUME V: THE CKA MOCK EXAMINATION SYSTEM

MOCK EXAM 4: FULL PROCTORED CERTIFICATION SIMULATION

EXAMINATION SPECIFICATIONS

- **Target Difficulty:** Full Proctored Certification Simulation (Realistic Exam Replica)
- **Time Allowed:** 120 minutes (2.0 hours)
- **Passing Score:** 66% (Total: 100%)
- **Number of Tasks:** 20 Performance-Based Tasks
- **Domain Weighting Alignment:**
    - Troubleshooting: 30% (Tasks 1, 2, 3, 4, 5, 6)
    - Cluster Architecture, Installation & Configuration: 25% (Tasks 7, 8, 9, 10, 11)
    - Services & Networking: 20% (Tasks 12, 13, 14, 15)
    - Workloads & Scheduling: 15% (Tasks 16, 17, 18)
    - Storage: 10% (Tasks 19, 20)

TASK 1: Troubleshoot and Repair a NotReady Node (Weight: 6%)

- **Domain:** Troubleshooting (30%)
- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Worker node `node-2` is in `NotReady` status.
- **Task Description:**
    1. Switch to context `k8s`.
    2. SSH to `node-2`.
    3. Identify why `kubelet` is in auto-restart loop.
    4. Fix configuration and ensure `node-2` returns to `Ready` status.
- **Expected Final Condition:** `kubectl get nodes` shows `node-2` in `Ready` status.
- **Solution:**

```
kubectl config use-context k8s
ssh node-2
sudo journalctl -u kubelet -n 30 --no-pager
# Identify issue: cgroup driver or swap
sudo swapoff -a
sudo systemctl restart kubelet
exit
kubectl get nodes node-2
```

TASK 2: Control Plane Static Pod Kube-Controller-Manager Failure (Weight: 5%)

- **Domain:** Troubleshooting (30%)
- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** ReplicaSets are not creating pods; KCM is dead.
- **Task Description:**
    1. Inspect `/etc/kubernetes/manifests/kube-controller-manager.yaml`.
    2. Correct invalid certificate path flag.
    3. Verify controller manager static pod restarts cleanly.
- **Expected Final Condition:** KCM is running and reconciles pending deployments.
- **Solution:**

```
sudo sed -i 's|/etc/kubernetes/pki/ca-broken.crt|/etc/kubernetes/pki/ca.crt|g' /etc/kubernetes/mani
```

TASK 3: Triage CrashLoopBackOff Pod with Previous Logs (Weight: 5%)

- **Domain:** Troubleshooting (30%)
- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Pod `auth-engine` in namespace `auth` in `CrashLoopBackOff`.
- **Task Description:**
    1. Inspect previous container logs.
    2. Fix missing environment variable `AUTH_KEY`.
- **Expected Final Condition:** Pod is `Running` with 0 restarts.
- **Solution:**

```
kubectl logs auth-engine -n auth --previous
kubectl set env pod auth-engine AUTH_KEY="production-secret" -n auth
```

TASK 4: Repair CoreDNS Loop Panic (Weight: 5%)

- **Domain:** Troubleshooting (30%)
- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** CoreDNS pods crashing with loop detection.
- **Task Description:**
    1. Configure kubelet on control plane to reference `/run/systemd/resolve/resolv.conf`.
    2. Restart kubelet and delete crashed CoreDNS pods.
- **Expected Final Condition:** CoreDNS pods show `2/2 Running`.
- **Solution:**

```
sudo sed -i 's|resolvConf: .*|resolvConf: /run/systemd/resolve/resolv.conf|g' /var/lib/kubelet/conf
sudo systemctl restart kubelet
kubectl delete pods -n kube-system -l k8s-app=kube-dns
```

TASK 5: Ingress Returns HTTP 503 (Missing Service Selector) (Weight: 5%)

- **Domain:** Troubleshooting (30%)
- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Ingress returns 503; backend Service has empty endpoints.
- **Task Description:**
    1. Fix the selector on Service `catalog-svc` to match application pods.
- **Expected Final Condition:** Ingress routes traffic cleanly.
- **Solution:**

```
kubectl patch svc catalog-svc -p '{"spec":{"selector":{"app":"catalog"}}}'
```

TASK 6: Triage Multi-Attach Volume Lock (Weight: 4%)

- **Domain:** Troubleshooting (30%)

- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Pod stuck in `ContainerCreating` with `Multi-Attach error`.
- **Task Description:**
    1. Inspect `VolumeAttachment` objects.
    2. Force delete the stuck old pod or delete the stale `VolumeAttachment`.
- **Expected Final Condition:** New pod mounts the volume and starts running.
- **Solution:**

```
kubectl get volumeattachments
kubectl delete volumeattachment <stale-attachment-id>
```

TASK 7: Kubeadm Control Plane Upgrade (Weight: 7%)

- **Domain:** Cluster Architecture (25%)
- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Primary control plane node at v1.30.2.
- **Task Description:**
    1. Upgrade `kubeadm` to `1.31.2-1.1`.
    2. Apply upgrade: `kubeadm upgrade apply v1.31.2`.
    3. Drain node, upgrade `kubelet` and `kubectl`, restart `kubelet`, uncordon.
- **Expected Final Condition:** Control plane node running `v1.31.2`.
- **Solution:**

```
sudo apt-mark unhold kubeadm
sudo apt-get update && sudo apt-get install -y kubeadm=1.31.2-1.1
sudo apt-mark hold kubeadm
sudo kubeadm upgrade plan
sudo kubeadm upgrade apply v1.31.2 -y
kubectl drain $(hostname) --ignore-daemonsets --delete-emptydir-data
sudo apt-mark unhold kubelet kubectl
sudo apt-get install -y kubelet=1.31.2-1.1 kubectl=1.31.2-1.1
sudo apt-mark hold kubelet kubectl
sudo systemctl daemon-reload && sudo systemctl restart kubelet
kubectl uncordon $(hostname)
```

TASK 8: etcd Snapshot Save and Status Verification (Weight: 6%)

- **Domain:** Cluster Architecture (25%)
- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Healthy etcd listening on port 2379.

- **Task Description:**
    1. Capture etcd snapshot to `/var/data/etcd-final-mock.db`.
    2. Output table status to verify non-zero size and revision.
- **Expected Final Condition:** Snapshot file created and verified.
- **Solution:**

```
sudo ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt --cert=/etc/kubernetes/pki/etcd/server.crt --key=/etc/ku
  snapshot save /var/data/etcd-final-mock.db
```

TASK 9: RBAC Role and RoleBinding with Specific Verbs (Weight: 4%)

- **Domain:** Cluster Architecture (25%)
- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Namespace `operations`.
- **Task Description:**
    1. Create ServiceAccount `deployer` in `operations`.
    2. Create Role `deploy-role` allowing `create`, `update`, `patch` on `deployments.apps`.
    3. Bind Role to ServiceAccount.
- **Expected Final Condition:** `deployer` can mutate deployments in `operations`.
- **Solution:**

```
kubectl create namespace operations
kubectl create serviceaccount deployer -n operations
kubectl create role deploy-role -n operations --verb=create,update,patch --resource=deployments.app
kubectl create rolebinding bind-deployer -n operations --role=deploy-role --serviceaccount=operatic
```

TASK 10: Deploy Application via Helm (Weight: 4%)

- **Domain:** Cluster Architecture (25%)
- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Chart repo `bitnami` added.
- **Task Description:**
    1. Install release `web-cache` using chart `bitnami/memcached` into namespace `cache`.
    2. Set replica count to `2`.

- **Expected Final Condition:** Release `web-cache` running in namespace `cache`.
- **Solution:**

```
helm install web-cache bitnami/memcached -n cache --create-namespace --set replicaCount=2
```

TASK 11: Declarative Kustomize Directory Application (Weight: 4%)

- **Domain:** Cluster Architecture (25%)
- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Directory `/opt/deploy/overlays/prod/`.
- **Task Description:**
    1. Apply Kustomize directory `/opt/deploy/overlays/prod/` to the cluster.
- **Expected Final Condition:** Resources deployed with production labels.
- **Solution:**

```
kubectl apply -k /opt/deploy/overlays/prod/
```

TASK 12: Gateway API HTTPRoute Traffic Management (Weight: 6%)

- **Domain:** Services & Networking (20%)
- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Gateway `edge-gw` exists in `gw-system`.
- **Task Description:**
    1. In namespace `sales`, create `HTTPRoute` named `sales-route` attached to `edge-gw`.
    2. Route path `/checkout` to Service `checkout-svc` on port 8080.
- **Expected Final Condition:** HTTPRoute reports `Accepted: True`.
- **Solution:**

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: sales-route
  namespace: sales
spec:
  parentRefs:
  - name: edge-gw
    namespace: gw-system
  rules:
  - matches:
    - path:
        type: PathPrefix
        value: /checkout
    backendRefs:
    - name: checkout-svc
      port: 8080
```

TASK 13: Ingress with TLS Termination (Weight: 5%)

- **Domain:** Services & Networking (20%)
- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Secret `tls-cert` exists. Service `app-svc` on port 80.
- **Task Description:**
    1. Author Ingress `app-ingress` with TLS terminating for `app.company.io` using Secret `tls-cert`.
- **Expected Final Condition:** Ingress created with TLS configuration.
- **Solution:**

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
spec:
  ingressClassName: nginx
  tls:
  - hosts:
    - app.company.io
    secretName: tls-cert
  rules:
  - host: app.company.io
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: app-svc
            port:
              number: 80
```

TASK 14: NetworkPolicy Default-Deny with Port 53 Egress (Weight: 5%)

- **Domain:** Services & Networking (20%)
- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Namespace `locked-ns`.
- **Task Description:**
    1. In namespace `locked-ns`, create NetworkPolicy denying all ingress, and allowing egress strictly on port 53 (UDP/TCP).
- **Expected Final Condition:** All ingress dropped; DNS egress permitted.
- **Solution:**

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: lock-and-dns
  namespace: locked-ns
spec:
  podSelector: {}
  policyTypes:
  - Ingress
  - Egress
  egress:
  - ports:
    - protocol: UDP
      port: 53
    - protocol: TCP
      port: 53
```

TASK 15: Expose Headless Service for StatefulSet (Weight: 4%)

- **Domain:** Services & Networking (20%)
- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** StatefulSet `elastic` exists in `logging`.
- **Task Description:**
    1. Create headless Service `elastic-discovery` with `clusterIP: None` on port 9200.
- **Expected Final Condition:** Headless service resolves to individual StatefulSet Pod IPs.
- **Solution:**

```
kubectl create service clusterip elastic-discovery --clusterip="None" --tcp=9200:9200 -n logging
```

TASK 16: Rolling Update and Immediate Rollback (Weight: 5%)

- **Domain:** Workloads & Scheduling (15%)
- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Deployment `api-fleet` running with 3 replicas.
- **Task Description:**
    1. Update image to `nginx:1.99-broken`.
    2. Observe rollout stall.
    3. Roll back immediately to the previous revision.
- **Expected Final Condition:** Deployment stabilized on healthy image.
- **Solution:**

```
kubectl set image deployment/api-fleet nginx=nginx:1.99-broken
kubectl rollout undo deployment/api-fleet
```

TASK 17: Pod Anti-Affinity Spreading Across Nodes (Weight: 5%)

- **Domain:** Workloads & Scheduling (15%)
- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Multi-node cluster.
- **Task Description:**
    1. Author Deployment `web-spread` with 2 replicas and `podAntiAffinity` requiring pods to land on distinct hostnames (`kubernetes.io/hostname`).
- **Expected Final Condition:** Each replica lands on a different worker node.
- **Solution:**

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-spread
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web-spread
  template:
    metadata:
      labels:
        app: web-spread
    spec:
      affinity:
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
          - labelSelector:
              matchExpressions:
              - key: app
                operator: In
                values:
                - web-spread
            topologyKey: kubernetes.io/hostname
      containers:
      - name: nginx
        image: nginx:1.27
```

TASK 18: ResourceQuota & Container LimitRange Configuration (Weight: 5%)

- **Domain:** Workloads & Scheduling (15%)
- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Namespace `staging`.
- **Task Description:**
    1. Apply ResourceQuota `quota-staging` with `hard.pods: "4"`.
    2. Apply LimitRange `limit-staging` with default CPU request `50m` and limit `100m`.
- **Expected Final Condition:** Limits enforced automatically on new pods.
- **Solution:**

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: quota-staging
  namespace: staging
spec:
  hard:
    pods: "4"
---
apiVersion: v1
kind: LimitRange
metadata:
  name: limit-staging
  namespace: staging
spec:
  limits:
  - default:
      cpu: 100m
    defaultRequest:
      cpu: 50m
    type: Container
```

TASK 19: PersistentVolume and PersistentVolumeClaim Binding (Weight: 5%)

- **Domain:** Storage (10%)
- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** Directory `/mnt/data/vol1` on worker node.
- **Task Description:**
    1. Create PV `pv-storage-1` of capacity 5Gi, accessMode `ReadWriteOnce`, storageClassName `local-vol`, hostPath `/mnt/data/vol1`.
    2. Create PVC `pvc-storage-1` requesting 2Gi on `local-vol`.
- **Expected Final Condition:** PV and PVC are `Bound`.
- **Solution:**

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-storage-1
spec:
  capacity:
    storage: 5Gi
  accessModes: [ "ReadWriteOnce" ]
  storageClassName: local-vol
  hostPath:
    path: /mnt/data/vol1
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-storage-1
spec:
  storageClassName: local-vol
  accessModes: [ "ReadWriteOnce" ]
  resources:
    requests:
      storage: 2Gi
```

TASK 20: Online PVC Volume Expansion (Weight: 5%)

- **Domain:** Storage (10%)
- **Context:** `kubectl config use-context k8s`
- **Initial Condition:** PVC `expand-claim` bound at 1Gi.
- **Task Description:**
    1. Dynamically expand `expand-claim` from `1Gi` to `3Gi`.
- **Expected Final Condition:** PVC capacity shows `3Gi`.
- **Solution:**

```
kubectl patch pvc expand-claim -p '{"spec":{"resources":{"requests":{"storage":"3Gi"}}}}'
```